

The final Keyword in Java

The `final` keyword in Java is a modifier that can be applied to **variables**, **methods**, and **classes**. It is used to impose restrictions on these entities to ensure their immutability, non-overriding, or non-inheritance, depending on the context.

1. `final` Keyword with Variables

When a variable is declared as `final`, its value cannot be changed after it is initialized. Once assigned, it acts as a constant.

Syntax:

```
final dataType variableName = value;
```

Example:

```
public class FinalVariableExample {
    public static void main(String[] args) {
        final int MAX_VALUE = 100; // Declaration and initialization
        System.out.println("MAX_VALUE: " + MAX_VALUE);

        // MAX_VALUE = 200; // Error: Cannot reassign a final variable
    }
}
```

Characteristics:

- A `final` variable must be initialized during declaration or within the constructor for instance variables.
- If not initialized at the time of declaration, it is called a **blank final variable** and must be initialized in the constructor.

Example of Blank Final Variable:

```
class FinalVariable {
    final int value; // Blank final variable

    // Constructor to initialize the blank final variable
    FinalVariable(int value) {
```

```

        this.value = value;
    }

    public void displayValue() {
        System.out.println("Final value: " + value);
    }
}

public class Main {
    public static void main(String[] args) {
        FinalVariable obj = new FinalVariable(50);
        obj.displayValue();
    }
}

```

2. final Keyword with Methods

When a method is declared as `final`, it cannot be overridden by subclasses. This is useful when you want to preserve the original behavior of a method.

Syntax:

```

final returnType methodName(parameters) {
    // Method body
}

```

Example:

```

class Parent {
    final void display() {
        System.out.println("This is a final method in the Parent class.");
    }
}

class Child extends Parent {
    // @Override
    // void display() { // Error: Cannot override a final method
    //     System.out.println("Trying to override a final method.");
    // }
}

public class FinalMethodExample {
    public static void main(String[] args) {
        Parent obj = new Parent();
        obj.display();
    }
}

```

```
}  
}
```

Characteristics:

- The `final` method can still be inherited by subclasses but cannot be modified.
 - It ensures the behavior of the method remains consistent across all subclasses.
-

3. `final` Keyword with Classes

When a class is declared as `final`, it cannot be extended. This is useful for creating immutable classes or classes that should not be modified.

Syntax:

```
final class ClassName {  
    // Class body  
}
```

Example:

```
final class FinalClass {  
    void display() {  
        System.out.println("This is a final class.");  
    }  
}  
  
// class SubClass extends FinalClass { // Error: Cannot inherit from a  
// final class  
// }  
  
public class FinalClassExample {  
    public static void main(String[] args) {  
        FinalClass obj = new FinalClass();  
        obj.display();  
    }  
}
```

Characteristics:

- A `final` class cannot have subclasses.
 - All methods in a `final` class are implicitly `final`.
-

4. `final` Keyword with Parameters

When a method parameter is declared as `final`, its value cannot be changed within the method body.

Syntax:

```
void methodName(final dataType parameter) {  
    // Method body  
}
```

Example:

```
public class FinalParameterExample {  
    void display(final int number) {  
        // number = 20; // Error: Cannot reassign a final parameter  
        System.out.println("Final parameter: " + number);  
    }  
  
    public static void main(String[] args) {  
        FinalParameterExample obj = new FinalParameterExample();  
        obj.display(10);  
    }  
}
```

Common Use Cases of `final`

1. Creating Constants:

- `final` variables combined with `static` are used to define constants.

```
public class Constants {  
    public static final double PI = 3.14159;  
    public static final int MAX_STUDENTS = 100;  
}
```

2. Preventing Method Overriding:

- Use `final` methods in base classes to ensure the implementation remains unchanged.

3. Preventing Inheritance:

- Use `final` for classes like utility or security-related classes to prevent modifications.

4. Immutable Classes:

- Combine `final` classes with `final` variables to create immutable classes.

```
final class ImmutableClass {  
    private final String name;  
  
    ImmutableClass(String name) {  
        this.name = name;  
    }  
  
    public String getName() {  
        return name;  
    }  
}
```

Key Points to Remember

Context	Behavior
Final Variable	Cannot be reassigned once initialized.
Final Method	Cannot be overridden in subclasses.
Final Class	Cannot be extended or inherited.
Final Parameter	Value cannot be modified within the method body.
